

phynotype: complete workflow

Contents

1	Introduction	2
2	Data preparation	2
2.1	Numeric data	2
2.2	Mixed-type data	2
3	Clustering algorithms	3
3.1	K-means	3
3.2	PAM	3
3.3	Hierarchical clustering	3
3.4	DBSCAN	4
3.5	Gaussian mixture models	5
3.6	K-prototypes	5
3.7	KMM (K-Mixed-Modes)	6
4	Accessors	6
5	Validation	7
5.1	Internal metrics	7
5.2	External metrics	8
5.3	Grid search	9
5.4	Per-cluster silhouette widths	9
6	Exploration	10
7	Prediction	13
8	Consensus meta-clustering	14
8.1	How it works	14
8.2	Usage	14
8.3	Validation and exploration of the consensus	15
9	Interpretation	16
9.1	The unsupervised interpretation problem	16
9.2	Global feature importance	17
9.3	Local explanations (LIME)	18
9.4	Ceteris paribus profiles	20
10	Complete pipeline example	21
11	References	21

1 Introduction

phynotype provides a unified grammar for unsupervised phenotyping. The full pipeline has six steps, each with a single entry-point function:

Step	Function	Purpose
1	<code>cluster()</code>	Fit a single clustering solution
2	<code>metacluster()</code>	Combine candidates into a consensus
3	<code>validate()</code>	Score a solution with internal/external metrics
4	<code>explore()</code>	Summarize sizes, profiles, and embeddings
5	<code>predict()</code>	Assign new observations
6	<code>feature_importance()</code> / <code>lime_explain()</code> / <code>ceteris_paribus()</code>	Interpret the clustering rule

All steps return structured S3 objects with `print()`, `summary()`, and `plot()` methods. This vignette is the canonical reference and walks through the complete workflow using the `iris` data set as a running example.

2 Data preparation

2.1 Numeric data

For purely numeric inputs, pass the matrix or data frame directly to `cluster()`. Centering and scaling are available via `center` and `scale`:

```
X <- iris[, 1:4] # 150 x 4 numeric matrix
head(X)
#>   Sepal.Length Sepal.Width Petal.Length Petal.Width
#> 1         5.1         3.5         1.4         0.2
#> 2         4.9         3.0         1.4         0.2
#> 3         4.7         3.2         1.3         0.2
#> 4         4.6         3.1         1.5         0.2
#> 5         5.0         3.6         1.4         0.2
#> 6         5.4         3.9         1.7         0.4
```

2.2 Mixed-type data

When the data contain categorical variables, two strategies are available depending on the clustering method:

One-hot encoding (for k-means, PAM, DBSCAN, GMM):

```
iris_mixed <- iris # 4 numeric + 1 factor column
iris_mixed$Group <- as.character(
  ifelse(iris$Sepal.Length > 5.8, "large", "small")
)
Xenc <- prepare_mixed_data(iris_mixed[, c(1:4, 6)], center = TRUE, scale = TRUE)
dim(Xenc)
#> [1] 150 6
```

Gower distance (for hierarchical methods):

```
d_gower <- mixed_distance(iris_mixed[, c(1:4, 6)])
class(d_gower)
#> [1] "dissimilarity" "dist"
```

The Gower dissimilarity between observations i and j is

$$d_G(i, j) = \frac{\sum_{f=1}^p w_{ijf} \delta_{ijf} s_{ijf}}{\sum_{f=1}^p w_{ijf} \delta_{ijf}},$$

where δ_{ijf} flags comparable pairs, w_{ijf} is an optional feature weight, and s_{ijf} is the feature-specific partial similarity (scaled absolute difference for numeric, mismatch indicator for categorical).

3 Clustering algorithms

3.1 K-means

Lloyd's algorithm (Lloyd, 1982) minimizes the total within-cluster sum of squares:

$$\min_c \sum_{j=1}^k \sum_{x_i \in C_j} \|x_i - \mu_j\|^2, \quad \mu_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i.$$

The algorithm alternates between nearest-centroid assignment and centroid recomputation until convergence. Multiple restarts (`nstart`) reduce sensitivity to random initialization.

```
fit_km <- cluster(X, method = "kmeans", k = 3, seed = 1)
fit_km
#> <cluster_fit>
#> Method: kmeans
#> Observations: 150
#> Clusters: 3
```

3.2 PAM

Partitioning Around Medoids (Kaufman and Rousseeuw, 1990) selects k actual observations (medoids) $m_1, \dots, m_k$ to minimize

$$\min_{m_1, \dots, m_k \in \mathcal{X}} \sum_{i=1}^n \min_g d(x_i, m_g).$$

Unlike k-means, medoids are always observed data points, making PAM more robust to outliers.

```
if (requireNamespace("cluster", quietly = TRUE)) {
  fit_pam <- cluster(X, method = "pam", k = 3)
  fit_pam
}
#> <cluster_fit>
#> Method: pam
#> Observations: 150
#> Clusters: 3
```

3.3 Hierarchical clustering

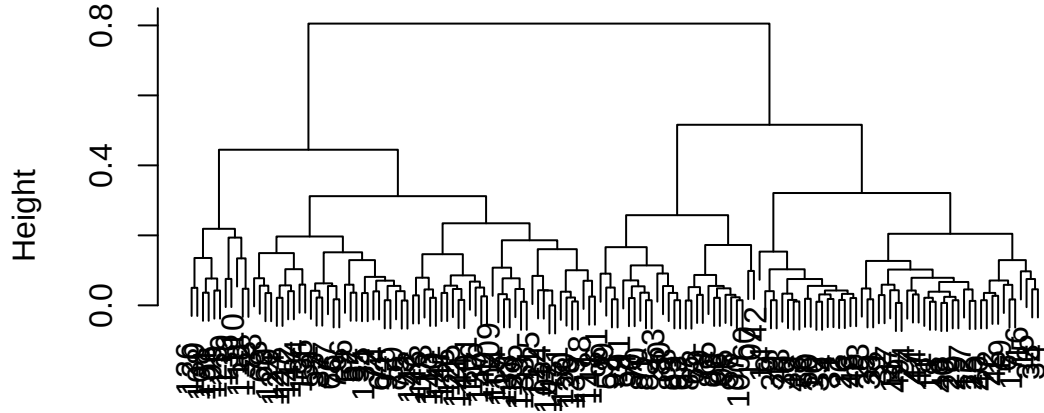
"hclust" and "agnes" build a dendrogram by successively merging the two closest clusters. The inter-cluster distance depends on the linkage:

Linkage	Formula
Complete	$\max_{a \in A, b \in B} d(a, b)$
Single	$\min_{a \in A, b \in B} d(a, b)$
Average (UPGMA)	$\frac{1}{ A B } \sum_{a, b} d(a, b)$
Ward	Minimize the increase in total within-cluster variance

The final partition is obtained by cutting the tree at height k .

```
# From a distance object (supports mixed data via mixed_distance())
d <- mixed_distance(X)
fit_hc <- cluster(d, method = "hclust", k = 3)
fit_hc
#> <cluster_fit>
#> Method: hclust
#> Observations: 150
#> Clusters: 3
plot_dendrogram(fit_hc)
```

Cluster dendrogram



AGNES (Kaufman and Rousseeuw, 1990) is the `cluster` package's implementation and additionally reports the agglomerative coefficient:

```
if (requireNamespace("cluster", quietly = TRUE)) {
  fit_ag <- cluster(d, method = "agnes", k = 3)
  fit_ag
}
#> <cluster_fit>
#> Method: agnes
#> Observations: 150
#> Clusters: 3
```

3.4 DBSCAN

DBSCAN (Ester et al., 1996) groups observations reachable through dense neighborhoods without requiring k . A point p is a *core point* when

$$|N_\epsilon(p)| = |\{q : d(p, q) \leq \epsilon\}| \geq \text{MinPts}.$$

Points not reachable from any core point are labeled *noise* (cluster 0).

```

if (requireNamespace("dbscan", quietly = TRUE)) {
  fit_db <- cluster(X, method = "dbscan", eps = 0.5, minPts = 5)
  fit_db
  # Noise points
  sum(clusters(fit_db) == 0)
}
#> [1] 17

```

3.5 Gaussian mixture models

A GMM models the data density as a weighted sum of Gaussians:

$$p(x) = \sum_{g=1}^k \pi_g \mathcal{N}(x \mid \mu_g, \Sigma_g).$$

Parameters (π_g, μ_g, Σ_g) are estimated by the EM algorithm (Fraley and Raftery, 2002). Hard labels come from the MAP rule $\hat{y}_i = \arg \max_g \pi_g \mathcal{N}(x_i \mid \mu_g, \Sigma_g)$; soft memberships are stored in `membership(fit)`.

```

if (requireNamespace("mclust", quietly = TRUE)) {
  fit_gmm <- cluster(X, method = "gmm", k = 3, seed = 1)
  fit_gmm
  # Soft membership probabilities
  head(membership(fit_gmm))
}
#>      [,1]      [,2]      [,3]
#> [1,] 1 4.916819e-40 3.345948e-29
#> [2,] 1 5.846760e-29 1.599452e-23
#> [3,] 1 2.486168e-33 2.724243e-25
#> [4,] 1 2.050996e-28 2.180764e-22
#> [5,] 1 8.283066e-42 8.710458e-30
#> [6,] 1 2.118466e-41 1.791499e-29

```

3.6 K-prototypes

K-prototypes (Huang, 1998) extends k-means to mixed data by combining squared Euclidean distance on numeric features with a Hamming mismatch penalty on categorical features:

$$D(x_i, p_g) = \underbrace{\sum_{j \in \mathcal{N}} (x_{ij} - \mu_{gj})^2}_{\text{numeric}} + \lambda \underbrace{\sum_{j \in \mathcal{C}} I(x_{ij} \neq \nu_{gj})}_{\text{categorical}}.$$

When `lambda = NULL`, λ is estimated automatically from the data.

```

if (requireNamespace("clustMixType", quietly = TRUE)) {
  fit_kp <- cluster(iris, method = "kproto", k = 3, seed = 1)
  fit_kp
  # Mixed prototypes (centroids + modes)
  prototypes(fit_kp)
}
#>      Sepal.Length Sepal.Width Petal.Length Petal.Width Species
#> cluster_1    0.07235294 -0.29262745    0.5067059    0.1340000 versicolor
#> cluster_2    0.77911565 -0.07365986    1.8154694    0.8333197 virginica
#> cluster_3   -0.83733333  0.37066667   -2.2960000   -0.9533333 setosa

```

3.7 KMM (K-Mixed-Modes)

KMM is `phynotype`'s native mixed-data algorithm. It minimizes the same weighted prototype distance as k-prototypes:

$$D(x_i, p_g) = \sum_{j \in \mathcal{N}} (x_{ij} - \mu_{gj})^2 + \lambda \sum_{j \in \mathcal{C}} I(x_{ij} \neq \nu_{gj}),$$

with $\lambda \geq 0$ estimated automatically (as the median numeric variance) when not supplied. KMM uses multiple random restarts and is designed for biostatistical applications where the number of categories is small relative to the number of observations.

```
iris_chr <- iris
iris_chr$Species <- as.character(iris_chr$Species)

fit_kmm <- cluster(iris_chr, method = "kmm", k = 3, seed = 1)
fit_kmm
#> <cluster_fit>
#> Method: kmm
#> Observations: 150
#> Clusters: 3
summary(fit_kmm)
#> Cluster fit summary
#> Method: kmm
#> Observations: 150
#> Clusters: 3
#> Sizes: 1=50, 2=50, 3=50
prototypes(fit_kmm)
#> Sepal.Length Sepal.Width Petal.Length Petal.Width Species
#> cluster_1      5.006      3.428      1.462      0.246 setosa
#> cluster_2      6.668      3.016      5.554      2.014 virginica
#> cluster_3      5.856      2.728      4.258      1.338 versicolor
```

4 Accessors

All `cluster_fit` and `metacluster_fit` objects share a uniform accessor interface:

```
# Integer assignment vector
head(clusters(fit_kmm))
#> [1] 3 3 3 3 3 3

# Numeric centroid matrix (k-means, hclust)
centers(fit_kmm)
#> Sepal.Length Sepal.Width Petal.Length Petal.Width
#> 1 0.05827957 -0.30894624 0.6355484 0.2345376
#> 2 1.00666667 0.01635088 1.9841053 0.8717193
#> 3 -0.83733333 0.37066667 -2.2960000 -0.9533333

# Named size vector
sizes(fit_kmm)
#> 1 2 3
#> 62 38 50

# Number of clusters
n_clusters(fit_kmm)
```

```
#> [1] 3

# Method label
method_used(fit_km)
#> [1] "kmeans"
```

5 Validation

`validate()` computes a table of internal metrics for any `cluster_fit` or `metacluster_fit`. External metrics are appended when `truth` labels are supplied.

5.1 Internal metrics

```
val <- validate(fit_km)
val$metrics_table
```

#>	metric	value	scale	direction
#> 1	silhouette	0.5528190	-1 to 1	higher is better
#> 2	calinski_harabasz	561.6277566	positive, unbounded	higher is better
#> 3	davies_bouldin	0.6619715	positive, unbounded	lower is better
#> 4	total_within	78.8514414	positive, unbounded	lower is better
#> 5	bootstrap_ari	0.9741540	<NA>	<NA>

The five internal metrics each capture a different aspect of cluster quality.

Silhouette width (Rousseeuw, 1987) measures how similar each observation is to its own cluster compared to the nearest alternative cluster. For observation i , let $a(i)$ be its mean distance to all other members of its cluster and $b(i)$ be its mean distance to the closest *other* cluster. The silhouette width is:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}.$$

This quantity lies in $[-1, 1]$. A value near +1 means i is well inside its cluster; near 0 means it sits on a cluster boundary; negative means it is closer on average to a neighboring cluster than to its own. `validate()` reports the mean over all observations — a global summary of cluster cohesion and separation simultaneously.

Calinski-Harabasz index (Caliński and Harabasz, 1974) treats clustering as an ANOVA-like problem: it asks how large the between-cluster dispersion is relative to the within-cluster dispersion, normalizing by the corresponding degrees of freedom:

$$CH = \frac{BSS/(k-1)}{WSS/(n-k)},$$

where BSS is the between-cluster sum of squares and WSS is the total within-cluster sum of squares. The ratio is unbounded above; higher values indicate tighter, more separated clusters. This index is most useful when comparing solutions with different values of k .

Davies-Bouldin index (Davies and Bouldin, 1979) penalizes clusters that are internally diffuse *and* close to their neighbors. For each cluster j , it finds the worst-case neighbor — the one that is both the most scattered and the closest — and averages these worst cases:

$$DB = \frac{1}{k} \sum_{j=1}^k \max_{l \neq j} \frac{s_j + s_l}{d(\mu_j, \mu_l)},$$

where s_j is the mean distance from the cluster- j members to centroid μ_j , and $d(\mu_j, \mu_l)$ is the Euclidean distance between centroids. Lower values are better; a perfect score of 0 would require clusters with zero internal scatter.

Total within-cluster sum of squares is the raw objective minimized by k-means, and is included as a direct measure of compactness:

$$\text{WSS} = \sum_{j=1}^k \sum_{i \in C_j} \|x_i - \mu_j\|^2.$$

It is most interpretable when compared across solutions with the same k — the “elbow” in a WSS-vs- k plot signals a natural cluster count.

Bootstrap ARI (Fang and Wang, 2012) addresses stability rather than geometric quality. It refits the same clustering model on B bootstrap resamples of the data and asks: how similar are the resulting partitions to the original? Formally,

$$\text{BootARI} = \frac{1}{B} \sum_{b=1}^B \text{ARI}(\hat{y}_{I^{(b)}}, \hat{y}^{(b)}),$$

where $I^{(b)}$ is the set of indices drawn in bootstrap b , $\hat{y}^{(b)}$ is the partition refitted on those indices, and ARI is evaluated only on the sampled observations. Values near 1 mean the cluster structure is robust to resampling; values below 0.6 suggest the solution is sensitive to the particular data points included.

5.2 External metrics

When reference labels are available:

```
val_ext <- validate(fit_km, truth = iris$Species)
val_ext$metrics_table
#>      metric      value      scale      direction
#> 1 silhouette 0.5528190 -1 to 1 higher is better
#> 2 calinski_harabasz 561.6277566 positive, unbounded higher is better
#> 3 davies_bouldin 0.6619715 positive, unbounded lower is better
#> 4 total_within 78.8514414 positive, unbounded lower is better
#> 5 ari 0.7302383 <NA> <NA>
#> 6 nmi 0.7582057 <NA> <NA>
#> 7 bootstrap_ari 0.9078536 <NA> <NA>
```

Adjusted Rand index (Hubert and Arabie, 1985) counts the proportion of observation pairs on which two labelings agree — both in the same cluster, or both in different clusters — and corrects that count for the level of agreement expected by chance alone. Given a contingency table (n_{ij}) with row sums a_i and column sums b_j :

$$\text{ARI}(U, V) = \frac{\sum_{ij} \binom{n_{ij}}{2} - E}{\frac{1}{2} \left[\sum_i \binom{a_i}{2} + \sum_j \binom{b_j}{2} \right] - E}, \quad E = \frac{\sum_i \binom{a_i}{2} \cdot \sum_j \binom{b_j}{2}}{\binom{n}{2}}.$$

A value of 1 indicates perfect agreement; the expected value under random labelings is 0; negative values are possible when agreement is worse than chance.

Normalized mutual information (Strehl and Ghosh, 2002) measures how much knowing one partition reduces uncertainty about the other, normalized so that the scale is comparable across different cluster counts:

$$\text{NMI}(U, V) = \frac{I(U; V)}{\sqrt{H(U) H(V)}},$$

where $I(U; V) = \sum_{u,v} p_{uv} \log(p_{uv}/p_u p_v)$ is mutual information and $H(U) = -\sum_u p_u \log p_u$ is Shannon entropy. Unlike ARI, NMI is not corrected for chance, but it is symmetric and bounded in $[0, 1]$, making it useful for reporting alongside ARI.

5.3 Grid search

To compare cluster counts, pass raw data directly:

```
grid_val <- validate(X, method = "kmeans", k = 2:6)
grid_val$metrics_table
```

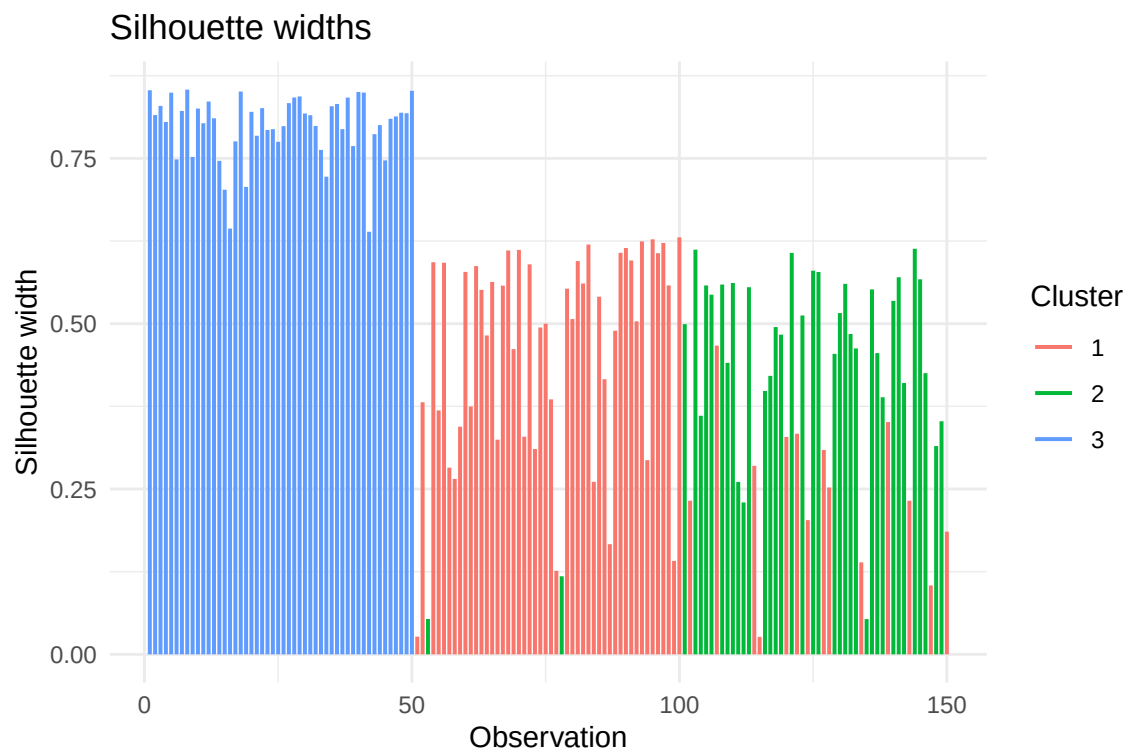
#>		metric	value	scale	direction	k
#> 1		silhouette	0.6810462	-1 to 1 higher is better		2
#> 2	calinski_harabasz	513.9245460	positive, unbounded higher is better			2
#> 3	davies_bouldin	0.4042928	positive, unbounded lower is better			2
#> 4	total_within	152.3479518	positive, unbounded lower is better			2
#> 5	bootstrap_ari	0.9946848		<NA>	<NA>	2
#> 6		silhouette	0.5528190	-1 to 1 higher is better		3
#> 7	calinski_harabasz	561.6277566	positive, unbounded higher is better			3
#> 8	davies_bouldin	0.6619715	positive, unbounded lower is better			3
#> 9	total_within	78.8514414	positive, unbounded lower is better			3
#> 10	bootstrap_ari	0.9583037		<NA>	<NA>	3
#> 11		silhouette	0.4980505	-1 to 1 higher is better		4
#> 12	calinski_harabasz	530.7658082	positive, unbounded higher is better			4
#> 13	davies_bouldin	0.7803070	positive, unbounded lower is better			4
#> 14	total_within	57.2284732	positive, unbounded lower is better			4
#> 15	bootstrap_ari	0.8973190		<NA>	<NA>	4
#> 16		silhouette	0.4887489	-1 to 1 higher is better		5
#> 17	calinski_harabasz	495.5414877	positive, unbounded higher is better			5
#> 18	davies_bouldin	0.8059652	positive, unbounded lower is better			5
#> 19	total_within	46.4461821	positive, unbounded lower is better			5
#> 20	bootstrap_ari	0.8679450		<NA>	<NA>	5
#> 21		silhouette	0.3648340	-1 to 1 higher is better		6
#> 22	calinski_harabasz	473.8506068	positive, unbounded higher is better			6
#> 23	davies_bouldin	0.9141580	positive, unbounded lower is better			6
#> 24	total_within	39.0399872	positive, unbounded lower is better			6
#> 25	bootstrap_ari	0.8153003		<NA>	<NA>	6

5.4 Per-cluster silhouette widths

Individual silhouette widths reveal which observations are well-assigned (positive width, comfortably inside their cluster) and which are borderline or potentially misclassified (width near zero or negative):

```
if (requireNamespace("cluster", quietly = TRUE)) {
  plot_silhouette(fit_km)
}
```

#> Warning: `aes\_string()` was deprecated in ggplot2 3.0.0.
#> i Please use tidy evaluation idioms with `aes()`.
#> i See also `vignette("ggplot2-in-packages")` for more information.
#> i The deprecated feature was likely used in the phynotype package.
#> Please report the issue to the authors.
#> This warning is displayed once per session.
#> Call `lifecycle::last\_lifecycle\_warnings()` to see where this warning was
#> generated.



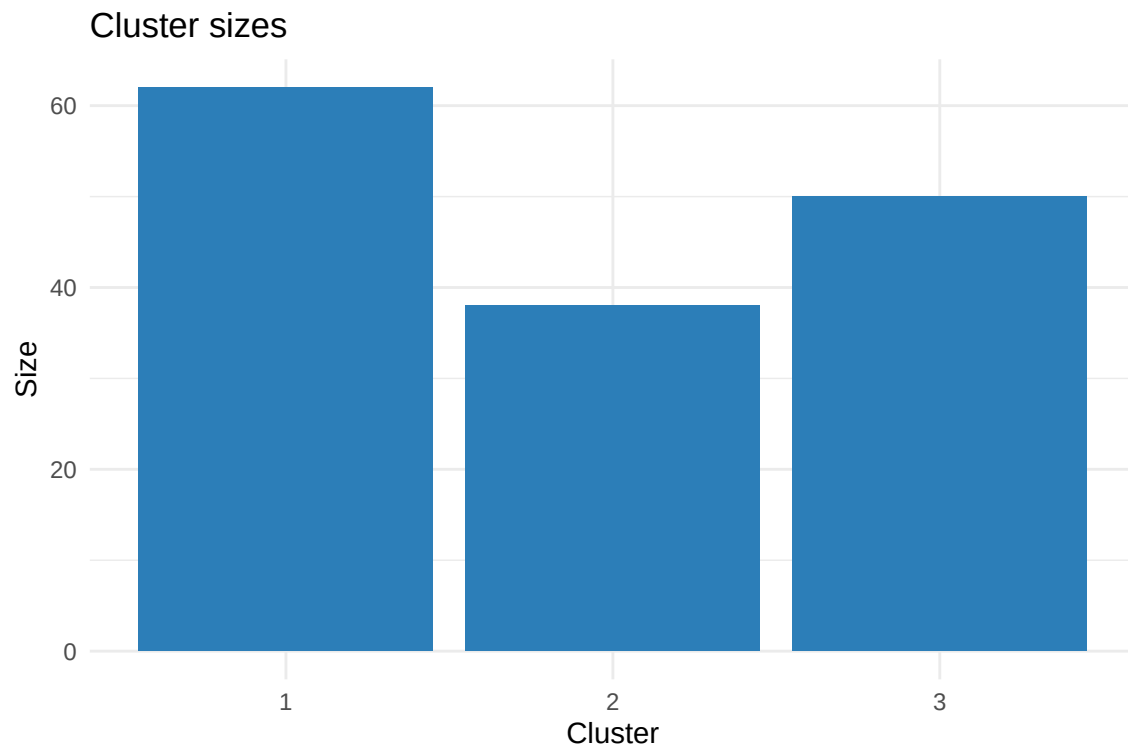
6 Exploration

`explore()` computes four structural summaries from a fitted clustering object.

```
exp <- explore(fit_km)
```

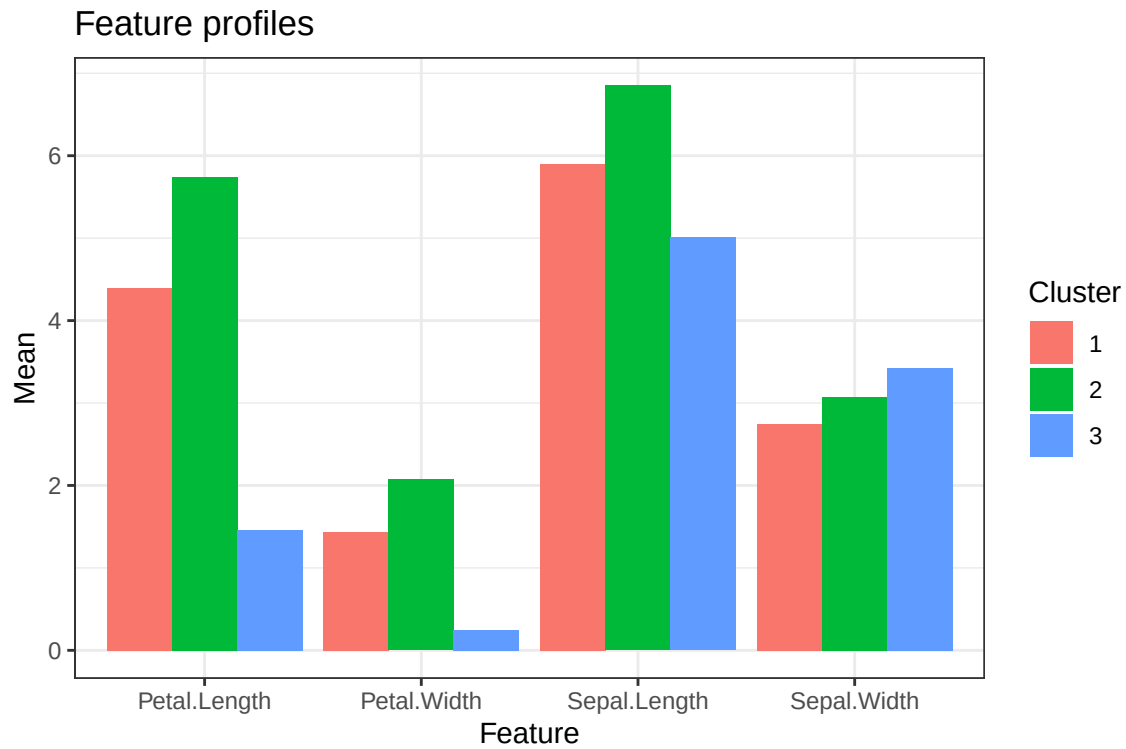
Cluster sizes:

```
exp$size_table  
#>   cluster size  
#> 1         1  62  
#> 2         2  38  
#> 3         3  50  
plot_cluster_sizes(fit_km)
```



Feature profiles (per-cluster mean, SD, median, min, max):

```
head(exp$feature_summary)
#>   cluster    feature      mean      sd median min max
#> 1      1 Sepal.Length 5.901613 0.4664101    5.9 4.9 7.0
#> 2      1  Sepal.Width 2.748387 0.2962841    2.8 2.0 3.4
#> 3      1 Petal.Length 4.393548 0.5088950    4.5 3.0 5.1
#> 4      1  Petal.Width 1.433871 0.2974997    1.4 1.0 2.4
#> 5      2 Sepal.Length 6.850000 0.4941550    6.7 6.1 7.9
#> 6      2  Sepal.Width 3.073684 0.2900924    3.0 2.5 3.8
plot_feature_profiles(exp)
```



Feature separation (eta-squared):

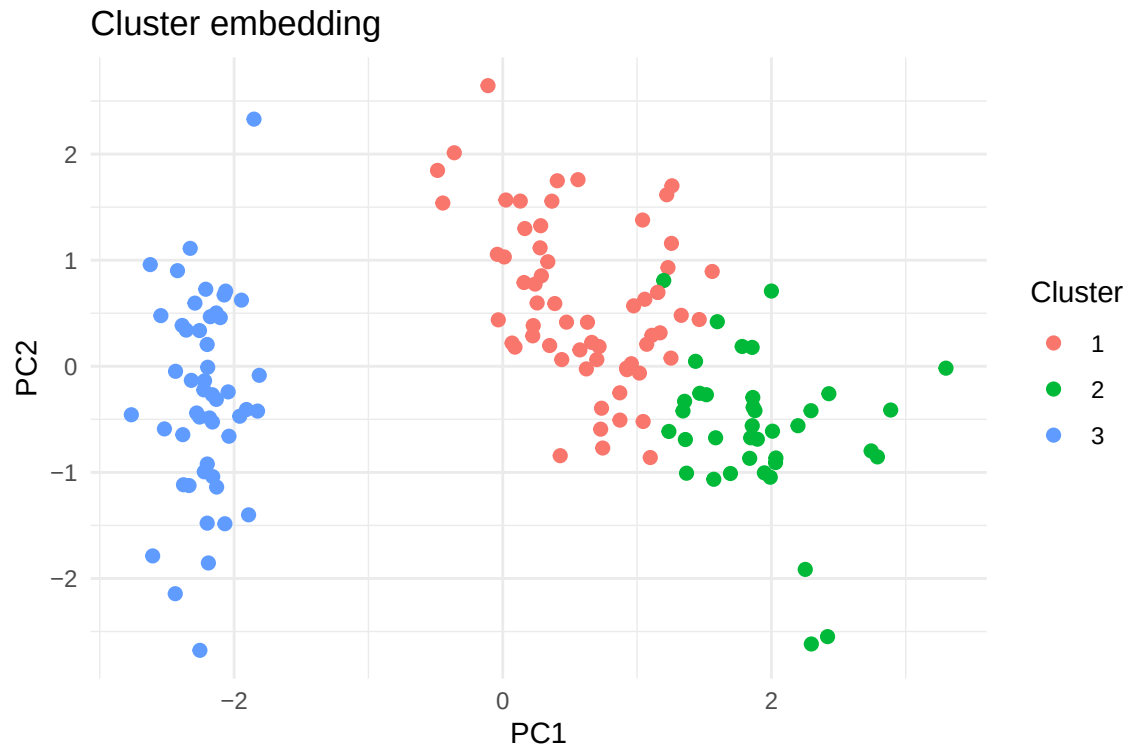
$$\eta_j^2 = \frac{SS_{B,j}}{SS_{T,j}} = \frac{\sum_{g=1}^k n_g (\bar{x}_{gj} - \bar{x}_j)^2}{\sum_{i=1}^n (x_{ij} - \bar{x}_j)^2}.$$

Values near 1 indicate features that strongly discriminate clusters.

```
exp$separation_table
#>               feature separation
#> Sepal.Length Sepal.Length 0.7220955
#> Sepal.Width  Sepal.Width 0.4521016
#> Petal.Length  Petal.Length 0.9437725
#> Petal.Width   Petal.Width 0.8978718
```

PCA embedding:

```
plot_clusters(fit_km)
```



7 Prediction

`predict()` assigns new observations to the nearest cluster center or prototype. The prediction rule is method-dependent: nearest centroid for k-means and hierarchical fits, nearest medoid for PAM, density-based for DBSCAN, and MAP for GMM.

```
new_obs <- data.frame(
  Sepal.Length = c(5.0, 6.5, 7.2),
  Sepal.Width  = c(3.5, 2.9, 3.2),
  Petal.Length = c(1.5, 4.5, 6.0),
  Petal.Width  = c(0.3, 1.5, 2.2)
)
```

```
pred <- predict(fit_km, new_obs)
pred$clusters
#> [1] 3 1 2
```

Distance to each cluster center:

```
head(pred$distances)
#>           1           2           3
#> [1,] 3.3220646 4.9735631 0.09787747
#> [2,] 0.6298878 1.4218273 3.64866825
#> [3,] 2.2488672 0.4707364 5.41083912
```

8 Consensus meta-clustering

When no single algorithm or value of k is clearly best, `metacluster()` pools candidate solutions into a consensus through co-association evidence accumulation (Fred and Jain, 2002).

8.1 How it works

For B candidate partitions, the co-association matrix $C \in [0, 1]^{n \times n}$ is

$$C_{ij} = \frac{1}{B} \sum_{b=1}^B I\{u_i^{(b)} = u_j^{(b)}\},$$

the proportion of candidates that co-cluster observations i and j . The consensus dissimilarity $D = 1 - C$ is then hierarchically clustered (average linkage) and the optimal k is chosen by maximizing the mean silhouette width:

$$k^* = \arg \max_{k \in \mathcal{K}} \bar{s}_k(D).$$

Ensemble stability is reported as the mean pairwise partition agreement (PPA):

$$\text{PPA}(U, V) = \frac{1}{\binom{n}{2}} \sum_{i < j} I\{(u_i = u_j) = (v_i = v_j)\}.$$

8.2 Usage

```
mfit <- metacluster(
  X,
  methods = c("kmeans", "pam", "hclust"),
  k       = 2:5,
  seed    = 1
)
mfit
#> <metacluster_fit>
#>   Methods: kmeans, pam, hclust
#>   Candidate fits: 12
#>   Final clusters: 5
```

Candidate partition table:

```
head(mfit$candidate_table)
#>   candidate method k n_clusters
#> 1      1 kmeans 2      2
#> 2      2 kmeans 3      3
#> 3      3 kmeans 4      4
#> 4      4 kmeans 5      5
#> 5      5   pam 2      2
#> 6      6   pam 3      3
```

Ensemble stability:

```
mfit$stability_summary
#>               metric mean_agreement min_agreement max_agreement
#> 1 pairwise_partition_agreement    0.8186128    0.6760889          1
```

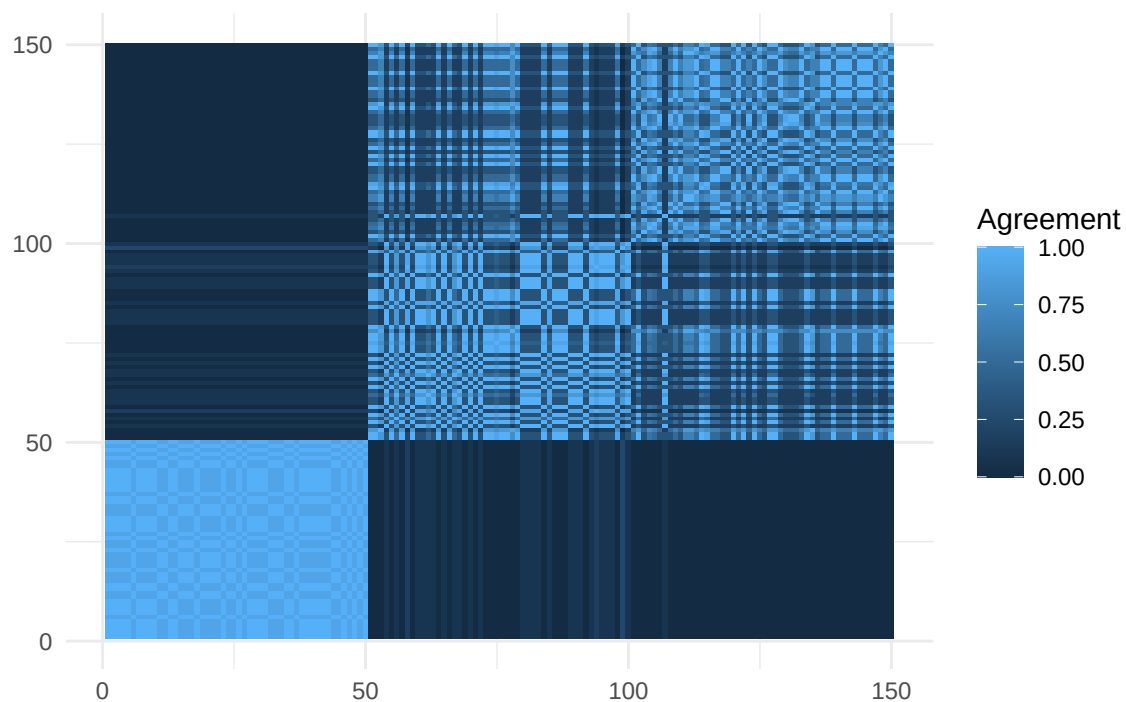
Consensus k selection scores:

```
mfit$selection_summary
#> k silhouette
#> 1 2 0.6566933
#> 2 3 0.7658527
#> 3 4 0.8603009
#> 4 5 0.9076378
```

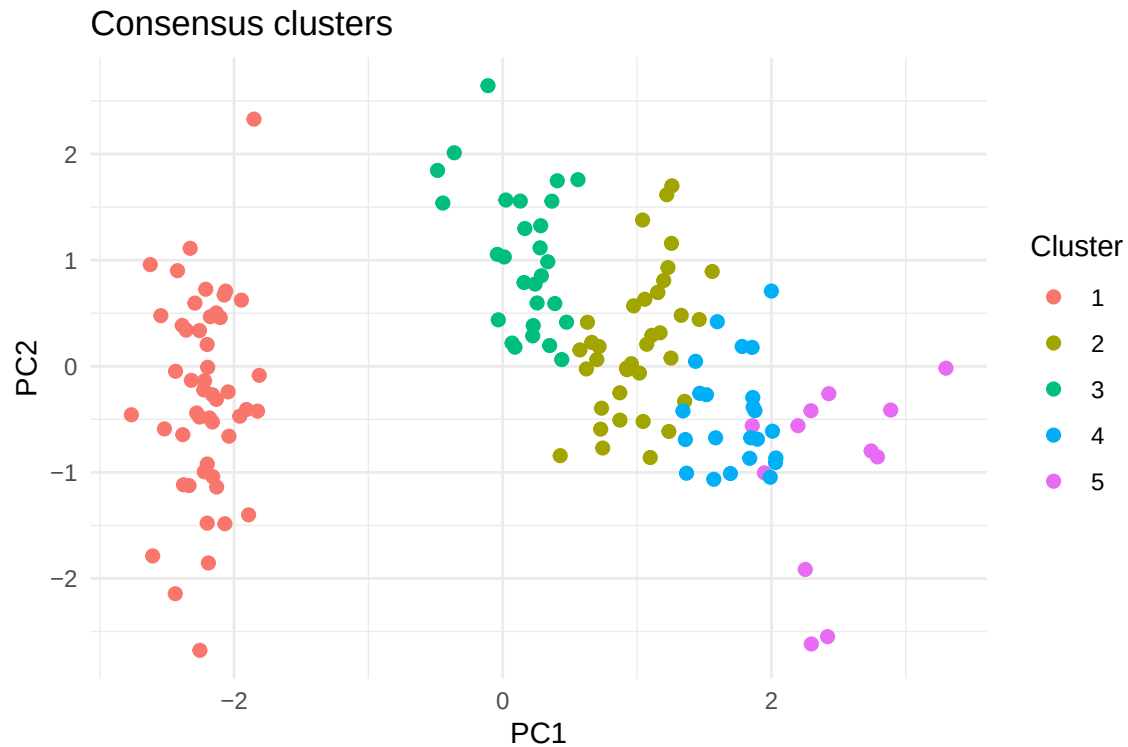
8.3 Validation and exploration of the consensus

```
validate(mfit)$metrics_table
#>      metric      value      scale      direction
#> 1 silhouette 0.4925619 -1 to 1 higher is better
#> 2 calinski_harabasz 494.0510734 positive, unbounded higher is better
#> 3 davies_bouldin 0.8168048 positive, unbounded lower is better
#> 4 pairwise_partition_agreement 0.8186128 <NA> <NA>
plot_coassoc(mfit)
```

Co-association matrix

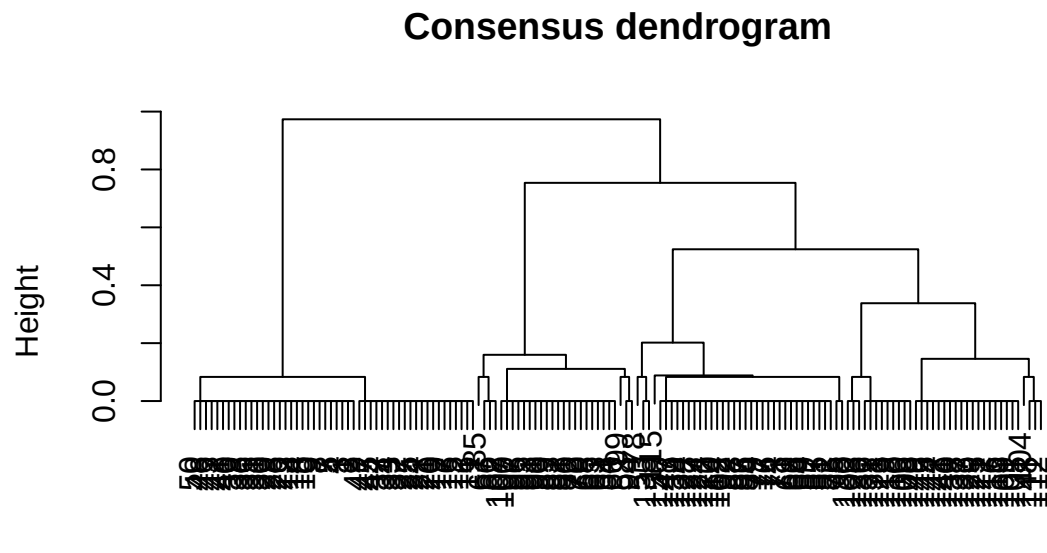


```
plot_consensus(mfit)
```



Consensus dendrogram:

```
plot_dendrogram(mfit)
```



9 Interpretation

9.1 The unsupervised interpretation problem

Classical interpretability methods — permutation importance, LIME, individual conditional profiles — were designed for supervised models where a response variable y is available. Clustering produces no such variable. The challenge is that we want to understand the clustering *rule* $f : \mathcal{X} \rightarrow C$, not the cluster structure in the data.

`phynotype` resolves this by treating the fitted model as a black-box predictor. Every supported method with a `predict()` implementation maps new observations to cluster labels (and optionally to distance or membership scores). This map is well-defined, deterministic, and can be probed exactly like any classifier. The three interpretability functions below operate exclusively through this map — they never look at the raw training data or the clustering criterion directly. As a result, the explanations describe **the decision boundary of the fitted model**, not the geometry of the original point cloud. This distinction matters: a feature can be highly informative for separating the data yet contribute little to the fitted boundary if the algorithm did not use it, and vice versa.

All three functions require a fit trained on row-by-feature data with prediction support ("`kmeans`", "`pam`", "`hclust`", "`agnes`", "`gmm`", "`kproto`", "`kmm`").

9.2 Global feature importance

What it answers: which features does the fitted clustering rule rely on most, globally across all observations?

The approach adapts permutation importance (Breiman, 2001; Fisher et al., 2019) to the unsupervised setting. The idea is simple: if a feature is important to the model, randomly shuffling its values across observations should disrupt the cluster assignments. If it is irrelevant, shuffling should leave assignments unchanged.

Concretely, let $\hat{c} = f(X)$ be the baseline partition produced by the fitted model on the evaluation data X . For each feature j , we independently permute its column M times and re-predict:

$$FI_j = \frac{1}{M} \sum_{m=1}^M \frac{1}{n} \sum_{i=1}^n I\left\{\hat{c}_i \neq f\left(X_{\pi(j)}^{(m)}\right)_i\right\}.$$

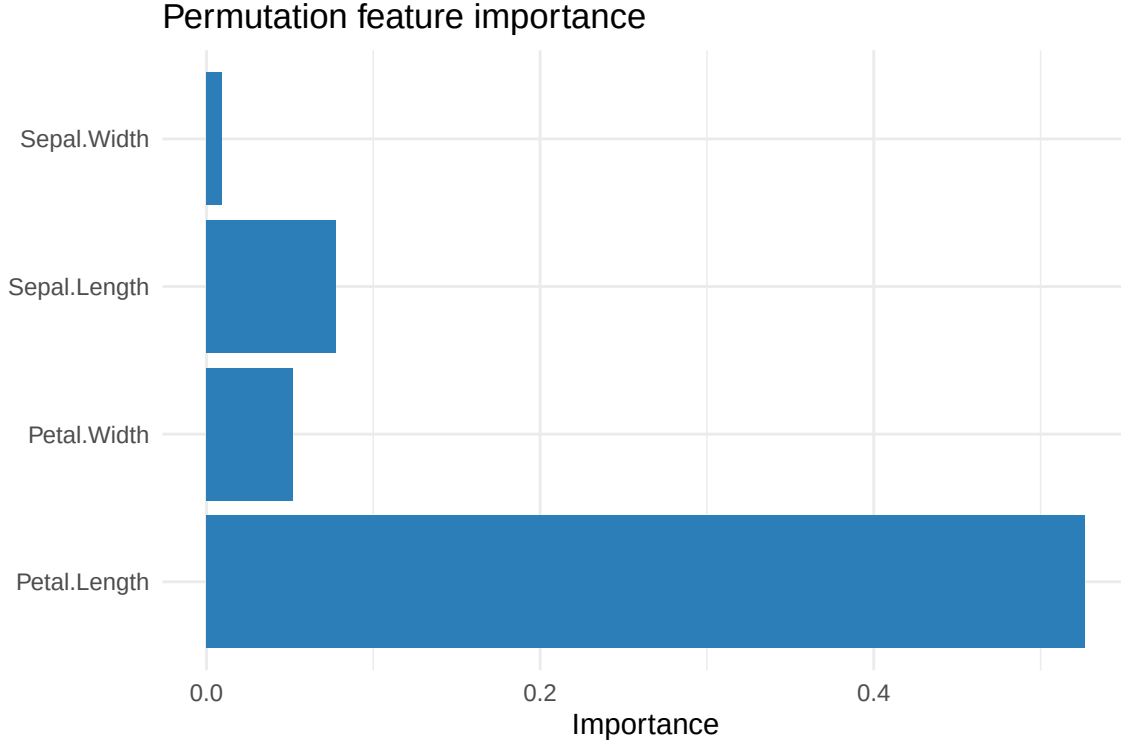
Here $X_{\pi(j)}^{(m)}$ is the data matrix with column j replaced by a random permutation of its values. The importance FI_j is the mean fraction of observations whose predicted cluster changes when column j is scrambled. A value of 0 means the model ignores feature j entirely; a value of 1 means every assignment changes when j is removed.

Two alternative metrics measure the same idea through internal scores rather than label changes. With `metric = "silhouette"`, importance is the mean drop in silhouette width after permutation:

$$FI_j^{\text{sil}} = \frac{1}{M} \sum_{m=1}^M \left[S(f, X) - S\left(f, X_{\pi(j)}^{(m)}\right) \right],$$

so a positive value means permuting j degrades within-cluster cohesion. With `metric = "total_within"`, it is the mean increase in within-cluster dispersion, capturing the same idea from a compactness perspective.

```
imp <- feature_importance(fit_km, n_repeats = 5, seed = 1)
imp$summary
#>      feature importance std_error n_repeats
#> 1 Petal.Length 0.526666667 0.005962848      5
#> 2 Sepal.Length 0.077333333 0.004521553      5
#> 3 Petal.Width 0.052000000 0.009285592      5
#> 4 Sepal.Width 0.009333333 0.003399346      5
plot(imp)
```



The bar chart ranks features by their mean importance. In this k-means fit on `iris`, `Petal.Length` and `Petal.Width` dominate — permuting either of them strongly disrupts which cluster each observation is sent to, confirming that the fitted boundary is primarily organised along the petal dimensions.

9.3 Local explanations (LIME)

What it answers: for a specific observation, which features pushed it into its assigned cluster, and in which direction?

Global importance tells us which features matter on average across all observations, but it cannot explain an individual assignment. LIME (Local Interpretable Model-agnostic Explanations; Ribeiro et al., 2016) addresses this by fitting a simple linear model locally around each observation being explained.

In the supervised setting LIME explains a classifier’s predicted class. Here the “classifier” is the clustering rule f . For each explained observation x , we define a target $g\{f(\cdot)\}$ that converts model output into a scalar:

- With `target = "cluster"` (default): $g\{f(z)\} = I\{f(z) = c\}$, a binary indicator of whether the perturbed sample z is predicted to belong to the same cluster c as the original observation. The local linear model then identifies which features explain membership in that cluster.
- With `target = "score"`: $g\{f(z)\}$ is the cluster-specific membership probability (for GMM) or a normalized radial similarity score derived from distances (for centroid/prototype methods). This gives a continuous response and is appropriate when soft assignment information is meaningful.

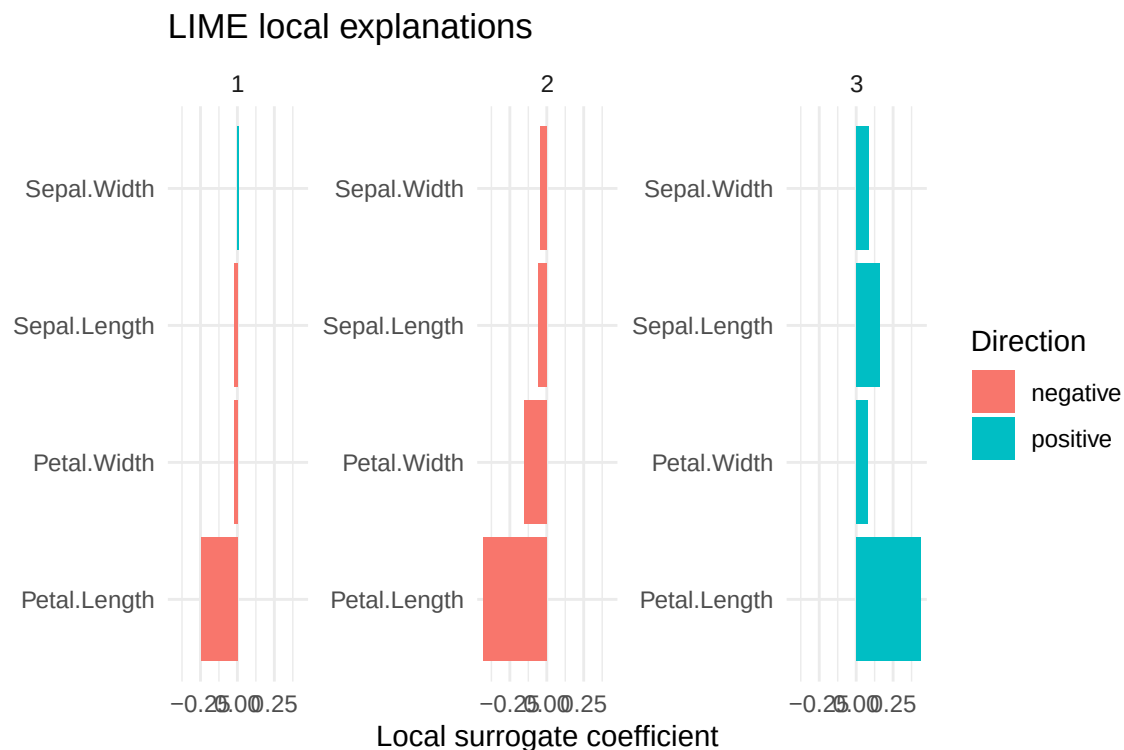
The LIME objective is to find a sparse linear surrogate $\hat{h}$ that approximates $g\{f(\cdot)\}$ in a weighted neighborhood around x :

$$\hat{h} = \arg \min_{h \in \mathcal{H}} \sum_{i=1}^N \pi_x(z_i) [g\{f(z_i)\} - h(z_i)]^2 + \Omega(h),$$

where $z_1, \dots, z_N$ are perturbed samples drawn around x (numeric features are perturbed by Gaussian noise calibrated to their training standard deviation; categorical features are randomly resampled from observed levels), and $\pi_x(z_i) = \exp(-\|x - z_i\|^2 / \sigma^2)$ is an exponential kernel that down-weights samples far from x . Sparsity $\Omega(h)$ is enforced operationally by returning only the top `n_features` absolute effects from the fitted weighted linear model.

The output is a signed coefficient for each feature: a positive coefficient means that increasing the feature value makes membership in cluster c more likely under the local linear approximation; a negative coefficient means the opposite. These are *local* effects — they describe the boundary in the immediate neighborhood of the explained observation and should not be extrapolated globally.

```
lx <- lime_explain(fit_km, iris[c(1, 51, 101), 1:4],
                  n_permutations = 100, n_features = 4, seed = 1)
lx$explanations[, c("observation", "feature", "estimate", "direction")]
#>   observation   feature   estimate direction
#> 1           1 Petal.Length -0.244296078 negative
#> 2           1 Sepal.Length -0.025137829 negative
#> 3           1 Petal.Width  -0.022188827 negative
#> 4           1 Sepal.Width  0.006827936 positive
#> 5           2 Petal.Length -0.427589581 negative
#> 6           2 Petal.Width  -0.148964717 negative
#> 7           2 Sepal.Length -0.058491693 negative
#> 8           2 Sepal.Width  -0.042806569 negative
#> 9           3 Petal.Length  0.435047397 positive
#> 10          3 Sepal.Length  0.160609418 positive
#> 11          3 Sepal.Width  0.081565890 positive
#> 12          3 Petal.Width  0.076892913 positive
plot(lx)
```



The plot shows one panel per explained observation. Reading the bars: observation 1 (setosa, cluster 1) is explained primarily by small `Petal.Length` pushing it away from the other clusters; observation 51 (versicolor) and observation 101 (virginica) are separated mainly along `Petal.Width` and `Petal.Length`, consistent with the global importance result.

9.4 Ceteris paribus profiles

What it answers: how does the predicted cluster assignment (or cluster score) change as one feature varies, holding everything else fixed at its observed value for a specific observation?

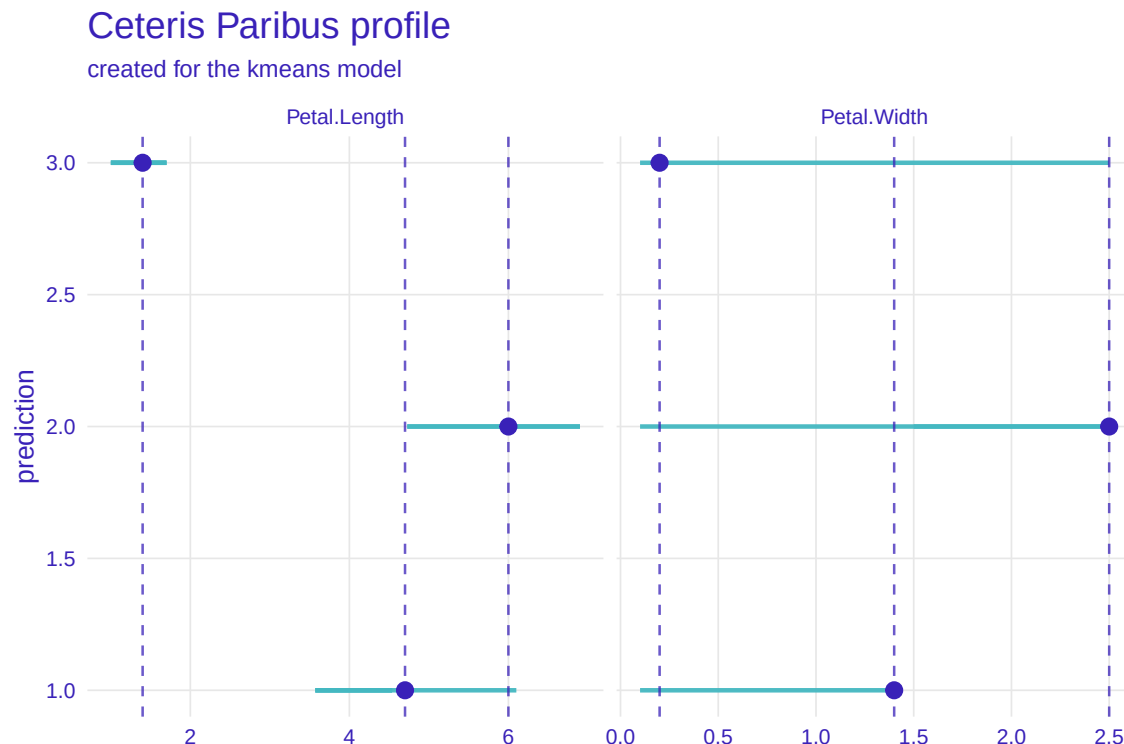
LIME gives a linear summary of the local boundary. Ceteris paribus (CP) profiles give the full nonlinear picture for each feature of interest. The name comes from the Latin for “all other things being equal”: for observation x_i and feature j , we slide j across a grid of values $z_1, \dots, z_G$ while clamping all other features at their observed values, then record the model’s prediction at each grid point:

$$\text{CP}_{i,j}(z) = g\{f(x_{i,-j}, z)\},$$

where $x_{i,-j}$ denotes the feature vector of observation i with feature j replaced by z , f is the fitted clustering rule, and g extracts either the cluster label (producing a step function, **target** = “cluster”) or a continuous cluster-specific score (**target** = “score”).

CP profiles answer the question: “if this particular patient had a different value for **Petal.Length**, would their phenotype assignment change, and at what threshold?” They are individual diagnostic curves — useful for understanding borderline cases and for communicating individual-level model behaviour to domain experts.

```
cp <- ceteris_paribus(  
  fit_km,  
  iris[c(1, 51, 101), 1:4],  
  features = c("Petal.Length", "Petal.Width"),  
  grid_size = 20  
)  
plot(cp)
```



Each panel shows one feature; each line corresponds to one explained observation. The dashed vertical line marks the observation’s actual feature value, and the dot marks its baseline prediction. Step changes in the profile (for **target** = “cluster”) reveal the exact feature thresholds at which the model switches cluster assignment — directly interpretable as phenotype decision boundaries.

10 Complete pipeline example

The full pipeline in compact form:

```
# 1. Fit
fit <- cluster(iris[, 1:4], method = "kmeans", k = 3, seed = 1)

# 2. Validate
val <- validate(fit, truth = iris$Species)

# 3. Explore
exp <- explore(fit)

# 4. Predict
pred <- predict(fit, iris[1:5, 1:4])

# 5. Interpret
imp <- feature_importance(fit, n_repeats = 10, seed = 1)
lx <- lime_explain(fit, iris[1:3, 1:4], n_permutations = 200, seed = 1)
cp <- ceteris_paribus(fit, iris[1:3, 1:4])

# 6. Consensus (when a single solution is uncertain)
mfit <- metacluster(iris[, 1:4],
                    methods = c("kmeans", "pam", "hclust"),
                    k = 2:5, seed = 1)
validate(mfit)$metrics_table
```

11 References

- Apley, D.W. and Zhu, J. (2020). Visualizing the effects of predictor variables in black box supervised learning models. *Journal of the Royal Statistical Society: Series B*, **82**(4), 1059–1086.
- Biecek, P. and Burzykowski, T. (2021). *Explanatory Model Analysis*. Chapman and Hall/CRC, Boca Raton. <https://ema.drwhy.ai/>
- Breiman, L. (2001). Random forests. *Machine Learning*, **45**(1), 5–32.
- Caliński, T. and Harabasz, J. (1974). A dendrite method for cluster analysis. *Communications in Statistics*, **3**(1), 1–27.
- Davies, D.L. and Bouldin, D.W. (1979). A cluster separation measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, **1**(2), 224–227.
- Ester, M., Kriegel, H.-P., Sander, J. and Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. *Proceedings of the 2nd ACM SIGKDD*, pp. 226–231.
- Fang, Y. and Wang, J. (2012). Selection of the number of clusters via the bootstrap method. *Computational Statistics and Data Analysis*, **56**(3), 468–477.
- Fisher, A., Rudin, C. and Dominici, F. (2019). All models are wrong, but many are useful: Learning a variable’s importance by studying an entire class of prediction models simultaneously. *Journal of Machine Learning Research*, **20**(177), 1–81.
- Fraley, C. and Raftery, A.E. (2002). Model-based clustering, discriminant analysis, and density estimation. *Journal of the American Statistical Association*, **97**(458), 611–631.

- Fred, A.L.N. and Jain, A.K. (2002). Data clustering using evidence accumulation. *Proceedings of the 16th International Conference on Pattern Recognition (ICPR'02)*, Vol. 4, pp. 276–280.
- Gosiewska, A. and Biecek, P. (2019). iBreakDown: Uncertainty of model explanations for non-additive predictive models. *arXiv:1903.11420*.
- Gower, J.C. (1971). A general coefficient of similarity and some of its properties. *Biometrics*, **27**(4), 857–874.
- Huang, Z. (1998). Extensions to the k-means algorithm for clustering large data sets with categorical values. *Data Mining and Knowledge Discovery*, **2**(3), 283–304.
- Hubert, L. and Arabie, P. (1985). Comparing partitions. *Journal of Classification*, **2**(1), 193–218.
- Kaufman, L. and Rousseeuw, P.J. (1990). *Finding Groups in Data: An Introduction to Cluster Analysis*. John Wiley & Sons, New York.
- Lloyd, S.P. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, **28**(2), 129–137.
- MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. *Proceedings of the 5th Berkeley Symposium on Mathematical Statistics and Probability*, Vol. 1, pp. 281–297.
- Murtagh, F. and Legendre, P. (2014). Ward’s hierarchical agglomerative clustering method: Which algorithms implement Ward’s criterion? *Journal of Classification*, **31**(3), 274–295.
- Ribeiro, M.T., Singh, S. and Guestrin, C. (2016). “Why should I trust you?”: Explaining the predictions of any classifier. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1135–1144.
- Rousseeuw, P.J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, **20**, 53–65.
- Strehl, A. and Ghosh, J. (2002). Cluster ensembles: A knowledge reuse framework for combining multiple partitions. *Journal of Machine Learning Research*, **3**, 583–617.
- Ward, J.H. (1963). Hierarchical grouping to optimize an objective function. *Journal of the American Statistical Association*, **58**(301), 236–244.